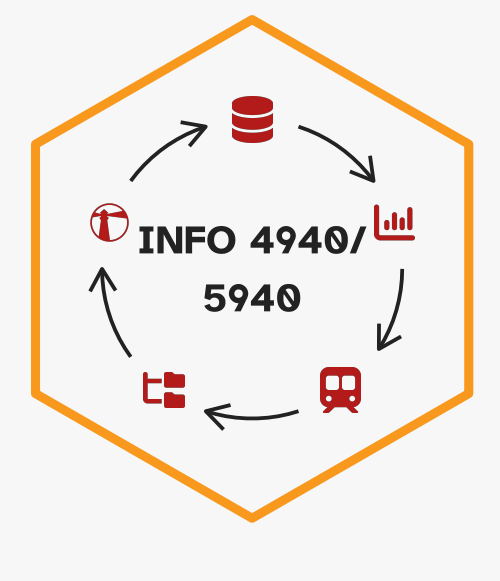




- Q
- Course information > Application exercises > Homework > HW 00 - Prefresher > HW 01 - Make a model > HW 02 - Build better data > HW 03 - Tune and evaluate models > HW 04 - Predict coffee preferences > HW 05 - Document and deploy models > HW 06 - Using LLMs to get stuff done > Project 01 > Project 02 > Extra credit > Tutorials

HW 06 - Using LLMs to get stuff done

MODIFIED
November 13, 2025

🔔 Important

This homework is due November 19 at 11:59pm ET.

Learning objectives

- Utilize LLMs to perform text classification tasks
- Evaluate the performance of different LLM models and prompt styles
- Design and implement a tutoring chatbot using LLMs and RAG techniques

Getting started

- Go to the [info4940-fa25](#) organization on GitHub. Click on the repo with the prefix **hw-06**. It contains the starter documents you need to complete the lab.
- Clone the repo and start a new workspace in Positron. See the [Homework 0 instructions](#) for details on cloning a repo and starting a new R project.

General guidance

🔍 Guidelines + tips

- Set your random seed to ensure reproducible results.
- Use [caching](#) to speed up the rendering process.
- Use **parallel processing** to speed up rendering time. Note that this works differently on different systems and operating systems, and it also makes it harder to debug code and track progress in model fitting. Use at your own discretion.

Remember that continuing to develop a sound workflow for reproducible data analysis is important as you complete this homework and other assignments in this course. There will be periodic reminders in this assignment to remind you to render, commit, and push your changes to GitHub. You should have **at least 3 commits with meaningful commit messages** by the end of the assignment.

🔍 Workflow + formatting

Make sure to

- Update author name on your document.
- Label all code chunks informatively and concisely.
- Follow consistent code style guidelines.
- Make at least 3 commits.
- Resize figures where needed, avoid tiny or huge plots.
- Turn in an organized, well formatted document.

⚠️ Presenting results of multiple models

For the love of all that is pure in this world, please consider how to present the results of your modeling efforts. Do not just rely on raw output from R to tell us what we need to know.

- Your plots should include an informative title, axes should be labeled, and careful consideration should be given to aesthetic choices.
- Maybe condense information into one or a handful of custom graphs.
- You can create simple formatted tables using [gt](#)/ [great_tables](#)

Part 1: Labeling legislative documents

Sample of legislative descriptions with policy categories

Description	Category Number	Policy Category Label
A bill to amend title 38, United States Code, to improve the disability compensation evaluation procedure of the Secretary of Veterans Affairs for veterans with mental health conditions related to military sexual trauma, and for other purposes.	3	Health
A bill to amend the Energy Policy and Conservation Act to improve energy efficiency of certain appliances and equipment, and for other purposes.	8	Energy
To establish the Maumee Valley National Heritage Area in Ohio and Indiana, and for other purposes.	8	Energy
To direct the Secretary of the Army to undertake a comprehensive review of the Corps of Engineers policy guidelines on vegetation management for levees, and for other purposes.	20	Public lands and water management
To authorize the Secretary of Veterans Affairs to make grants to eligible educational institutions to provide child care services on campus.	6	Education

The dataset `data/leg_lite.feather` contains descriptions of legislative bills from the U.S. Congress. Our goal is to classify these legislative bill descriptions into one of the [Comparative Agendas Project's major policy categories](#).

Comparative Agendas Project major policy categories

Category Number	Policy Category Label
1	Macroeconomics
2	Civil rights, minority issues, civil liberties
3	Health
4	Agriculture
5	Labor and employment
6	Education
7	Environment
8	Energy
9	Immigration
10	Transportation
11	Law, crime, family issues
12	Social welfare
13	Community development and housing issues
14	Banking, finance, and domestic commerce
15	Defense
16	Space, technology, and communications
17	Foreign trade
18	International affairs and foreign aid
19	Government operations
20	Public lands and water management

🔔 Differences from the codebook

The data uses the 2014 version of the codebook which differs from the most recent codebook published online. You can find the [major policy topics summarized on page 65](#). The original codebook also does not include anything for number 11. This dataset fills in the gap by shifting all the larger numbers down by 1 (i.e., 12 becomes 11, 13 becomes 12, etc.).

In this homework we will use a large language model (LLM) to classify these descriptions into one of the policy categories.

Exercise 1

Generate labels using an LLM. We will test several different LLM models and prompt styles to see how they affect classification performance. Specifically, we will implement a 3x3 design with three distinct models:

- GPT 4.1
- GPT 5-nano
- GPT 5

and three distinct prompts:

- **Naive and simple** - provide the LLM with the legislative description and ask it to return the policy category number. The prompt must be fewer than 100 words (not including the legislative description itself).
- **Explicit code/label values** - in addition to the naive/simple instructions, provide the LLM with the legislative description and the full list of policy categories (both number and label).
- **Detailed and reasoning** - use a prompt generator (e.g., [Claude prompt generator](#)) to create an improved prompt for the task using best practices for prompt engineering.

You will test all nine combinations of model and prompt on the training set of legislative descriptions. For each combination, generate predicted policy categories for all the training set observations. If possible, keep track of token usage and cost for each chat.

You should write your code to generate the predictions in the provided standalone R or Python script.¹ Store your prompts in the provided template files. Your script should export a data file which includes all the predictions from the nine model/prompt combinations along with the true labels.

🔍 Recommendations to make this easier

- You can write a separate system prompt with generic instructions for the LLM and a user prompt with the legislative description, or you can write a single prompt combining both pieces of information and pass it as the user prompt.
- Write a standalone function to generate predictions. At minimum, it should likely take three arguments:
 1. LLM model
 2. Prompt
 3. Dataset of legislative descriptions
- Use the `chat_structured()` method to ensure consistent output from the LLM. At minimum you will want to return the policy category number. It may also be useful to return the reasoning used by the LLM if available.
- Test your function on a small set of rows (e.g. five observations) first to ensure it works as intended.
- Use **batch processing** to minimize costs. Remember it can take up to 24 hours for batch jobs to finish, so leave yourself enough time to complete the assignment. Parallel processing is twice as expensive, so expect to be rewarded on your evaluation if you minimize costs by using batch processing.

Exercise 2

Evaluate the performance of LLM-generated labels. Using the predictions generated in Exercise 1, evaluate the performance of each model/prompt combination using the following metrics:

- Accuracy
- *F*-measure
- Sensitivity
- Specificity

Summarize the results in a table and/or figure to compare performance across models and prompts. Discuss which model/prompt combinations performed best and why you think that is the case.

If you were able to retain token usage/cost information, estimate the cost of each model/prompt combination. Discuss which model/prompt combinations provide the best value for money.

Part 2: Build an INFO 4940/5940 tutor chat bot

Exercise 3

Design and implement a tutoring chatbot for this class. Design and implement a chatbot that can assist students in this class with questions about course content, assignments, and other relevant topics. Your chatbot should be able to:

- Answer questions about course topics covered in lectures and readings.
- Provide guidance on assignments and projects.
- Answer questions about course policies from the syllabus.
- Offer study tips and resources.
- Help students implement the coding techniques required for the assignments.

At minimum, your chatbot must:

- Be created using the [shinychat](#) framework.
- Be able to answer questions from users in R or Python. Unlike [Ezra](#), it does not have to support both languages.
- Use a system prompt that clearly defines the role of the chatbot as an INFO 4940/5940 tutor and provides clear instructions on what it should and should not do.
- Include one or more RAG knowledge stores using appropriate resources (e.g. course syllabus, lecture notes, assignment instructions, relevant textbooks or articles).

Feel free to go above and beyond to make your chatbot more useful, engaging, and effective. This could include customizations to the user interface, additional features, or enhanced functionality.

You must deploy your app using [Posit Connect Cloud](#) and include a visible link to the publicly accessible app in `hw-06.qmd`. You will need to create an account on Posit Connect Cloud but the free tier should be sufficient for this assignment.

🔍 Deployment using Posit Cloud Connect

See [this article on deployment](#) for instructions on using Posit Publisher to deploy from within Positron, and ask questions if you need further guidance!

Keep in mind in order for your app to work on Posit Connect Cloud, you will need to ensure that all required resources (e.g. data files, images, CSS) are stored with your app or a subfolder. Likewise, you will need to store your LLM provider API key on Posit Connect Cloud as well. When you deploy the app, you can set environment variables in the deployment settings. Best practice is to generate a new API key specifically for this deployment, rather than reusing the API key from your local machine/Posit Workbench.

Generative AI (GAI) self-reflection

As stated in the [syllabus](#), include a written reflection for this assignment of how you used GAI tools (e.g. what tools you used, how you used them to assist you with writing code), what skills you believe you acquired, and how you believe you demonstrated mastery of the learning objectives.

➡ Render, commit, and push one last time.

📝

Make sure that you commit and push all changed documents and your Git pane is completely empty before proceeding.

Wrap up

Submission

- Go to <http://www.gradescope.com> and click *Log in* in the top right corner.
- Click *School Credentials* → *Cornell University NetID* and log in using your NetID credentials.
- Click on your *INFO 4940/5940* course.
- Click on the assignment, and you'll be prompted to submit it.
- Mark all the pages associated with exercise. All the pages of your homework should be associated with at least one question (i.e., should be "checked").

Grading

- Exercise 1: 20 points
- Exercise 2: 5 points
- Exercise 3: 25 points
- GAI self-reflection: 0 points
- Total: 50 points

Footnotes